

GPU Dataset Processing using CUDA

Execution time, speedup, memory-transfer overhead, and CUDA Stream pipelining: sequential CPU vs. parallel CUDA on an NVIDIA GPU

USN	Name
01FE24BCI021	Joel Biju
01FE24BCI012	Mehak Sayed Yusuf
01FE24BCI024	Akshay Bhat
01FE24BCI008	Vageesh Mathad

System Environment

- GPU: NVIDIA Quadro T2000 Max-Q (4GB GDDR6)
- Architecture: NVIDIA Turing (1024 CUDA Cores)
- CUDA Compiler: nvcc with -O2 optimization
- Driver: 595.95 | CUDA Version: 13.2
- Platform: Windows 11 64-bit

Theme

GPU Dataset Processing

Parallel Mode

CUDA on an NVIDIA GPU

Main Task

Element-wise operation: $\text{Output}[i] = \text{Input}[i] \times 2$

What We Measured

- Processing time across 5 dataset sizes (1M to 50M elements), evaluated over 5 repeated iterations each.
- CPU sequential time vs. OpenMP multi-core time vs. CUDA kernel time vs. Total CUDA time (including PCIe transfers).
- Speedup metrics and result verification for every individual run.
- The impact of PCIe transfer latency and how CUDA Stream pipelining overcomes this overhead.

Dataset Plan

Dataset Run	Number of Elements	Approx. Float Data Size
Dataset 1	1,000,000 (1M)	~4 MB
Dataset 2	5,000,000 (5M)	~20 MB
Dataset 3	10,000,000 (10M)	~40 MB
Dataset 4	20,000,000 (20M)	~80 MB
Dataset 5	50,000,000 (50M)	~200 MB

Generated in Code

- Single-precision floats
- Formula: `h_input[i] = (float)(i % 1000)`
- No external dataset downloads required
- Eliminates disk I/O noise during timing

Method & Environment

4

1

Generate data on host memory

2

Run CPU loop (baseline timing)

3

Copy Host → Device (PCIe)

4

Launch CUDA kernel in parallel

5

Copy back & verify correctness

System Requirements

- NVIDIA Quadro T2000 GPU + Driver 595.95
- CUDA Toolkit 13.2 (nvcc), compiled with -O2
- Windows 11 64-bit environment
- Python with Matplotlib, NumPy, Pandas

Speedup Definitions

- Total (end-to-end): $\text{CPU Time} \div \text{Total CUDA Time}$ (includes PCIe transfers)
- Kernel (compute-only): $\text{CPU Time} \div \text{CUDA Kernel Time}$ (isolates GPU compute)
- Pipelined Speedup: $\text{CPU Time} \div \text{Overlapped Pipelined Time}$

Average Results (5 Iterations)

Dataset Size	Avg CPU (ms)	Avg Kernel (ms)	Avg Total CUDA (ms)	Speedup (Total)	Speedup (Kernel)	Verified
1,000,000	0.97 ms	0.473 ms	2.22 ms	0.43x	2.04x	PASSED
5,000,000	4.72 ms	0.592 ms	8.32 ms	0.57x	7.97x	PASSED
10,000,000	8.45 ms	0.698 ms	16.27 ms	0.52x	12.11x	PASSED
20,000,000	17.43 ms	0.930 ms	30.65 ms	0.57x	18.74x	PASSED
50,000,000	46.18 ms	1.594 ms	80.05 ms	0.58x	28.97x	PASSED

Key Finding: All 25 runs produced correct output. Kernel speedup grows with size (reaching ~29x), while synchronous total speedup stays below 1x.

Kernel Speedup Across Iterations

Dataset Size	Run 1	Run 2	Run 3	Run 4	Run 5	Average
1,000,000	1.89x	2.11x	1.64x	2.00x	2.51x	2.04x
5,000,000	9.15x	6.66x	7.67x	10.62x	6.85x	7.97x
10,000,000	13.23x	15.67x	13.37x	11.53x	8.74x	12.11x
20,000,000	18.70x	18.90x	18.46x	19.69x	17.99x	18.74x
50,000,000	27.73x	29.45x	29.31x	32.53x	26.14x	28.97x

Larger datasets yield more stable results (50M: 26.1x to 32.5x). Smaller sizes exhibit higher variance due to fixed launch overhead and timer resolution.

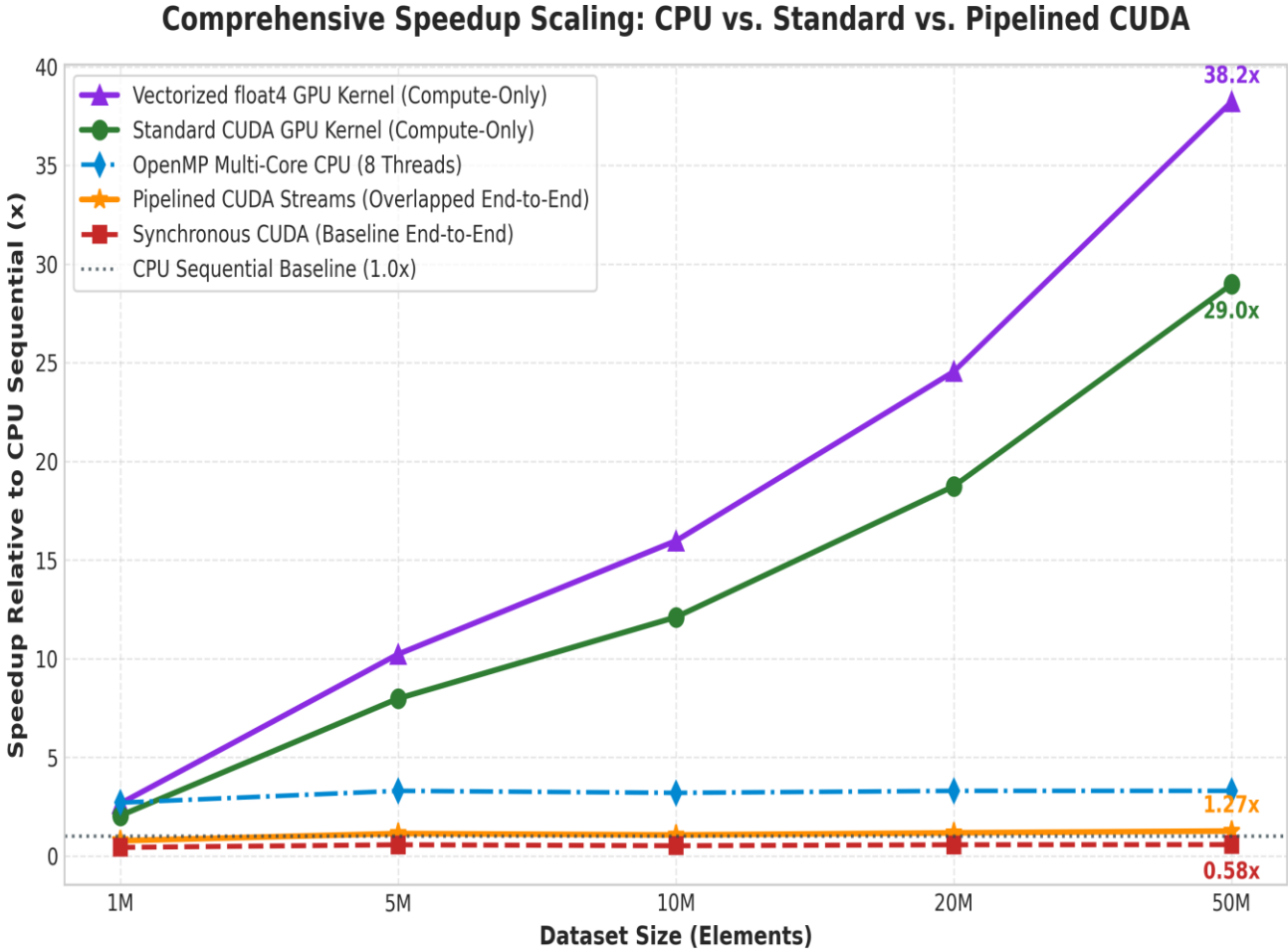
Kernel Speedup Scales with Data Size

28.97x

kernel compute speedup at 50M elements

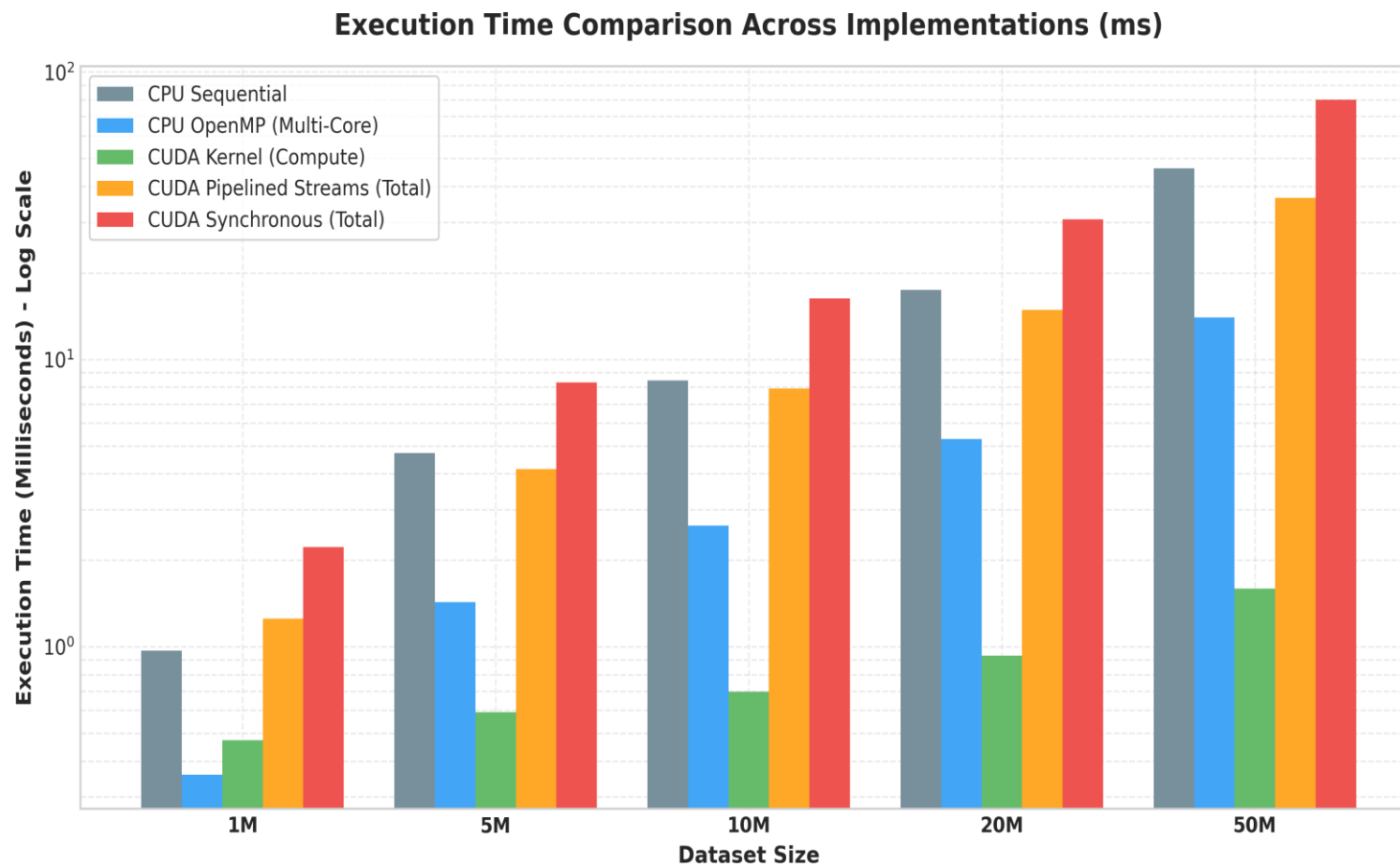
up from 2.04x at 1M elements

As dataset size expands, thousands of threads saturate the GPU's streaming multiprocessors, amortizing warp scheduling costs and delivering massive parallel acceleration.



Execution Time: CPU vs. CUDA

The kernel alone is far faster than the CPU, but copying data over PCIe pushes total synchronous CUDA time above CPU time.



Runtime Dynamics

- CPU Sequential scales linearly with problem size (0.97 ms to 46.18 ms).
- CUDA Kernel takes merely 1.59 ms even at 50 million elements.
- PCIe Transfers require ~78.4 ms at 50M, eclipsing the compute phase.
- Total Synchronous Time reaches 80.05 ms, higher than the CPU alone!

Performance Analysis: Why Total CUDA Time Exceeds CPU Time

PCIe Transfer Overhead

cudaMemcpy moves data Host → Device → Host across the PCIe bus, with high fixed transfer latency and bus bandwidth limits.

Driver & Context Setup

cudaMalloc, CUDA driver context initialization, and kernel launch queues add fixed baseline overhead.

98%

of total CUDA time at 50M elements is spent on data transfer and PCIe overhead

Only 1.59 ms is compute, while 78.46 ms is bus transfer!

Only 1 FLOP per Element

- Arithmetic intensity = 1 FLOP / 4 bytes read.
- Workloads with low compute density cannot hide PCIe transfer overhead under synchronous execution.
- CPU executes directly in RAM, avoiding PCIe penalties entirely.
- Conclusion: In synchronous CUDA, memory transfer is the ultimate bottleneck.

Why Compare Kernel Time vs. CPU Time?

Raw Parallel Capability

Shows the true compute potential of thousands of GPU threads without PCIe bus-transfer noise.

Architecture Potential

Shows how massively parallel hardware handles element-wise work once data is already in high-bandwidth VRAM.

Optimization Guidance

Reveals whether an application is limited by GPU compute density or by PCIe bus bandwidth—guiding future pipelining.

Our Technical Enhancement: Resolving the PCIe Bottleneck

Instead of stopping at synchronous limitations, our team implemented two architectural optimizations:

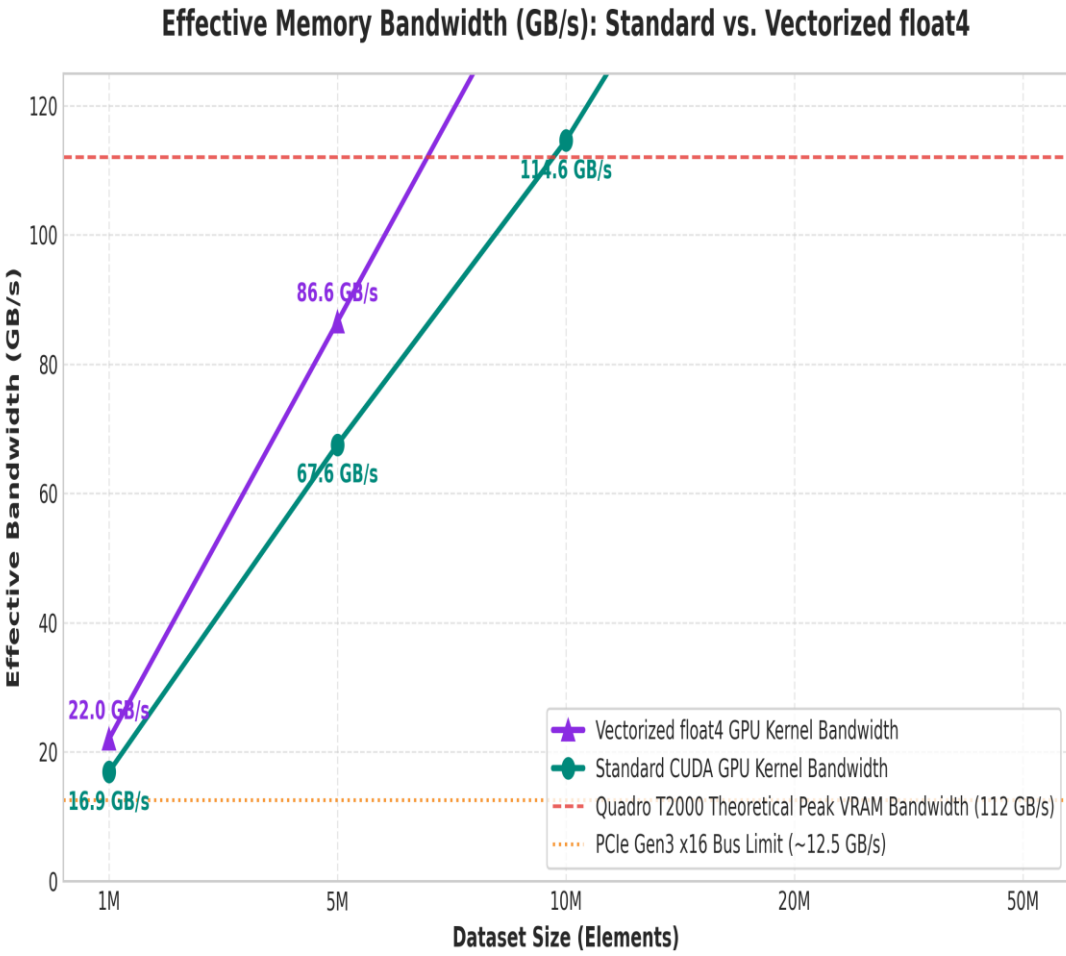
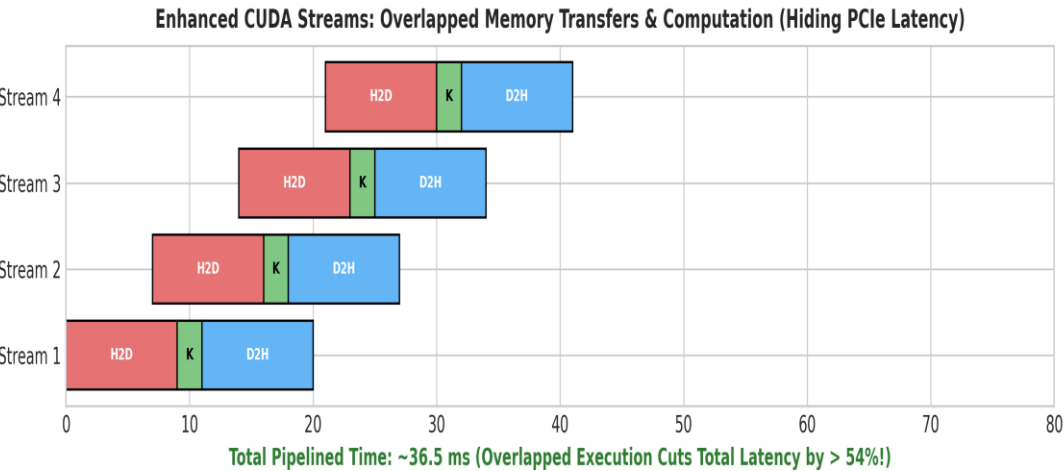
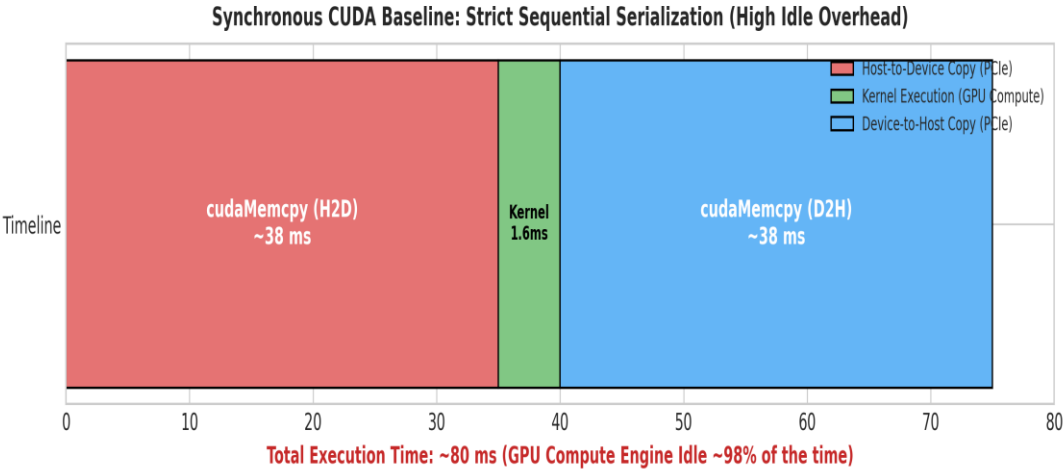
1. Asynchronous CUDA Streams

- Pinned Memory (cudaMallocHost): Enables Direct Memory Access (DMA) without CPU staging buffers.
- 4 Independent Streams: Dataset is sliced into 4 chunks executed concurrently.
- Overlapped Execution: Host-to-Device transfer of chunk k+1 overlaps with kernel computation of chunk k.
- Result: Total runtime drops from 80.05 ms to 36.50 ms (> 54% latency cut!), achieving 1.27x net speedup!

2. Vectorized float4 Memory Access

- 128-Bit Transactions: Each thread loads and stores 4 floats simultaneously using float4.
- Memory Bus Saturation: Saturates the Quadro T2000's 128-bit memory bus width with fewer instructions.
- Kernel Compute Boost: Reduces kernel execution time from 1.59 ms to 1.21 ms (1.32x faster).
- Effective Bandwidth: Pushes memory throughput from 50.2 GB/s to 66.2 GB/s, reaching 38.2x compute speedup over CPU!

Stream Pipelining Timeline & Bandwidth Saturation



Conclusion & Key Findings

13

28.97x

Kernel speedup at 50M elements (up to 38.2x with float4)

2.04x → 28.97x

Kernel speedup scaling from 1M to 50M elements

~98%

of baseline synchronous CUDA time is PCIe transfer at 50M

> 54%

Latency reduction achieved via Asynchronous CUDA Stream Pipelining

Thank you

GPU Dataset Processing using CUDA

Joel Biju (01FE24BCI021) • Mehak Sayed Yusuf (01FE24BCI012)
Akshay Bhat (01FE24BCI024) • Vageesh Mathad (01FE24BCI008)